

Mini AlphaGo

1. 实验介绍

1.1 实验内容

黑白棋（Reversi），也叫苹果棋，翻转棋，是一个经典的策略性游戏。一般棋子双面为黑白两色，故称“黑白棋”。因为行棋之时将对方棋子翻转，则变为己方棋子，故又称“翻转棋”（Reversi）。棋子双面为红、绿色的称为“苹果棋”。它使用 8x8 的棋盘，由两人执黑子和白子轮流下棋，最后子多方为胜方。随着网络的普及，黑白棋作为一种最适合在网上玩的棋类游戏正在逐渐流行起来。中国主要的黑白棋游戏站点有 Yahoo 游戏、中国游戏网、联众游戏等。

1.2 实验要求

- 使用 **【蒙特卡洛树搜索算法】** 实现 miniAlphaGo for Reversi。
- 使用 Python 语言。
- 算法部分需要自己实现，不要使用现成的包、工具或者接口。

1.3 注意事项

- Python 与 Python Package 的使用方式，可在右侧 [API 文档](#) 中查阅。
- 在与人类玩家对奕时，运行环境将等待用户输入座标，此时代码将处于 While...Loop 回圈中，请务必输入'Q' 离开，否则将持续系统将等待（hold）。
- 当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』-『Restart Kernel』）。

2. 实验思路

2.1 黑白棋分析

2.1.1 游戏规则

棋局开始时黑棋位于 E4 和 D5，白棋位于 D4 和 E5，如图所示。



1. 黑方先行，双方交替下棋。
2. 一步合法的棋步包括：
 - 在一个空格新落下一个棋子，并且翻转对手一个或多个棋子；
 - 新落下的棋子必须落在可夹住对方棋子的位置上，对方被夹住的所有棋子都要翻转过来，可以是横着夹，竖着夹，或是斜着夹。
 - 夹住的位置上必须全部是对手的棋子，不能有空格；
 - 一步棋可以在数个（横向，纵向，对角线）方向上翻棋，任何被夹住的棋子都必须被翻转过来，棋手无权选择不翻某个棋子。
3. 如果一方没有合法棋步，也就是说不管他下到哪里，都不能至少翻转对手的一个棋子，那他这一轮只能弃权，而由他的对手继续落子直到他有合法棋步可下。
4. 如果一方至少有一步合法棋步可下，他就必须落子，不得弃权。
5. 棋局持续下去，直到棋盘填满或者双方都无合法棋步可下。
6. 如果某一方落子时间超过 1 分钟 或者 连续落子 3 次不合法，则判该方失败。

2.1.2 策略解读

经过资料查找，常见的游戏策略有：

- **多子策略：**该策略的核心是每一步都翻转尽可能多的棋子。虽然表面上看起来是占据优势，但由于缺乏长远考虑，这种策略在对弈后期容易被对方翻盘。
- **稳定子-位置策略：**角上的棋子无法被翻转，因为它们不会被对手的两个棋子夹住，因此角上的棋子被称为“稳定子”。一旦占据了角，相邻的同色棋子也会变得稳定，从而不易被翻转。
- **行动力策略：**该策略旨在减少对方的行动选择，通过将己方棋子紧缩在对方的包围圈中，诱导对手落子到不利的位置（如星位），以便己方可以迅速占角。

除了上述策略，黑白棋还包括其他策略，比如爬边策略、楔入策略、余裕手策略和非平

衡边策略。但在开发 AI 时，最常用的还是**极大极小算法**。这种算法模拟人类的思维方式，通过在每一步中找到对自己最有利的走法来提高盘面评分，同时降低对方的得分。算法以己方得分为正，对方得分为负，将每个回合的评分累加，选择得分最高的路径。这种方法广泛应用于各类博弈，如象棋、五子棋等，其关键区别在于不同游戏中的棋盘评估函数。

在本实验中，我们将采用**蒙特卡洛树搜索算法**来进一步分析和解决问题。

2.2 蒙特卡洛树算法 MCTS

蒙特卡洛树搜索是一种用于决策过程，特别是在游戏玩法中选择最佳行动的算法。它通过构建和更新一棵搜索树来进行决策，这棵树从当前的游戏状态开始，并通过一系列步骤逐步扩展。MCTS 可以大致分为四个阶段：选择、扩展、模拟和反向传播。

1. 选择（selection）：算法从搜索树的根节点开始，依据一定的策略挑选子节点进行深入探索，直到抵达一个叶子节点或是一个拥有未被探索子节点的位置。这个过程依赖于节点已有的信息，如访问次数和累积奖励，来决定最值得探索的方向。
2. 扩展（expansion）：一旦到达了一个非终止状态的叶子节点 L，如果该节点还有未被探索的后继状态，则随机选择其中一个未曾探索过的后继节点 M 进行扩展。这意味着在搜索树中为该新状态创建一个新的节点。
3. 模拟（simulation）：从新扩展的节点 M 出发，使用一种称为“默认策略”的方法继续模拟游戏过程，直至达到游戏的终局。此阶段通常采用较为简单的策略，例如完全随机地选择动作，因为这里的目的是快速评估新扩展节点的价值，而不是做出最优选择。
4. 反向传播（Back Propagation）：模拟结束后，根据模拟的结果（例如最终得分），更新从新节点 M 到根节点路径上所有节点的信息。这包括增加每个节点的访问次数以及调整它们的累计奖励，以反映新的结果。这些更新有助于改进未来的搜索决策。

通过重复上述四个阶段，蒙特卡洛树搜索能够逐渐集中于最有潜力的选项，从而提高其推荐决策的质量。

2.3 上限置信区间 UCB1 算法

2.3.1 UCB1 算法解读

UCB1 算法是一种基于置信区间的多臂老虎机问题的解决方案，它通过计算每个臂的置信上界来决定在实验中应该选择哪个臂进行尝试。在蒙特卡洛树搜索（MCTS）框架下，UCB1 算法帮助确定在有限的搜索步骤中应该探索哪个子节点，以便更有效地接近最优策略。简而言之，UCB1 算法通过平衡探索（尝试未充分了解的选项）和利用（选择当前最佳选项）来指导决策过程。UCB1 算法中的置信区间计算方式为：

$$UCB1 = Q(s, a) + c * \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

其中， $Q(s, a)$ 是状态 s 和动作 a 的经验平均奖励值， $N(s)$ 是状态 s 在树中被访问的次数， $N(s, a)$ 是在状态 s 执行动作 a 的次数， c 是一个常数，用于平衡探索和开发的权衡， $\sqrt{\cdot}$ 是平方根函数， $\ln$ 是自然对数函数。UCB 算法中的常数项是 c ，这个参数扮演着调节探索与利用平衡的角色。当 c 值较大时，算法倾向于进行更多的探索，即尝试那些尚未充分了解的选项；而当 c 值较小时，算法则更偏向于利用已知信息，即选择那些目前看起来最优的选项。在 UCB1 这一特定变体中， c 被固定为 1。然而，在 UCB 算法的一些衍生版本中，通过调整 c 的值，可以优化算法的表现，以达到更好的探索与利用之间的平衡。简而言之， c 参数是 UCB 算法中用于微调以适应不同场景的关键因素。

这个公式实际上是在探索 (exploration) 和利用 (exploitation) 之间寻求一个平衡点。如果我们只考虑公式中的第一项，那么算法将完全基于当前的最佳表现来做出选择，这是一种贪婪策略。但这种策略可能会导致算法过早地陷入局部最优解，而忽略了全局最优解的可能性。第二项的存在是为了解决这个问题。它代表了对一个节点不确定性的度量。如果一个节点被探索的次数很少，我们对其平均回报的估计就不够准确，因此不确定性很高，置信区间也就相应地变大。在这种情况下，我们不应该完全相信当前的平均回报就是这个节点的真实表现，因此需要通过选择这个节点来获取更多的信息。因此，第二项可以被看作是衡量我们对一个节点了解程度的指标。了解得越少，第二项的值就越大。通过引入这个指标，算法在面对那些我们了解不足的节点时，即使它们的平均回报看起来不高，也会倾向于选择它们，以增加对这些节点的了解。这样，算法就表现出了一种“好奇心”，它倾向于探索那些我们还不够了解的领域，以期发现可能的更优解。

UCB 计算代码如下：

```
Python
def get_ucb(self, ucb_param):
    if self.visits == 0:
        return sys.maxsize # 未访问的节点 ucb 为无穷大

    if self.ucb_strategy == "ucb1":
        # UCB1 公式
        explore = math.sqrt(2.0 * math.log(self.parent.visits) /
float(self.visits))
```

```

elif self.ucb_strategy == "ucb2":

    # UCB2 公式

    explore = math.sqrt(2.0 * math.log(1 + self.parent.visits) / (1
+ float(self.visits)))

    now_ucb = self.reward / self.visits + ucb_param * explore

    return now_ucb

```

2.3.2 其他 UCB 算法介绍

(1) UCB2 算法

$$UCB2(s, a) = Q(s, a) + c * \sqrt{\frac{\ln(1 + N(s))}{1 + N(s, a)}}$$

UCB2 算法在 UCB1 的基础上引入了一个平滑因子，其值设为 1，这个因子的作用是调整状态-动作对的置信区间，使其更加保守。具体来说，这个平滑因子能够降低那些尚未充分探索的状态-动作对的置信上限，促使算法更倾向于去探索这些未知的领域。

同时，UCB2 算法通过减少对已经探索过的状态-动作对的探索频率，来优化探索行为。这意味着，对于那些已经有一定了解的状态-动作对，UCB2 算法会减少对它们的进一步探索，从而避免在已经相对了解的领域中进行过多的、可能没有太大价值的探索。这样的调整有助于算法更高效地分配探索资源，专注于那些最需要信息的状态-动作对，以期发现更优的策略。简而言之，UCB2 通过调整置信区间和探索倾向，旨在实现更加智能和有针对性的探索。

(2) 不同的置信区间上限计算方式

$$UCB = Q(s, a) + c * \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

UCB1 算法是 UCB 家族中的基础版本，它通过计算每个子节点的置信区间上限来决定下一步的行动。然而，UCB1 的一个局限性在于它对置信区间上限中的常数项采取了固定不变的假设，这在某些情况下可能会限制算法的表现。

为了克服这一局限，研究者们开发了多种 UCB 算法的改进版本，这些版本通过不同

的方法来计算置信区间上限，以期提升算法的整体效能。以下是一些流行的改进方案：

- UCB-Tuned：此版本通过动态调整置信区间上限中的常数项来优化 UCB1，这种调整基于之前对各个子节点的选择次数。
- UCB-V：这个版本在计算置信区间上限时，考虑了方差的影响，从而提高了 UCB1 的性能。
- UCB-Improved：此版本通过为不同的子节点设置不同的置信区间上限，以增强 UCB1 的性能。

这些改进版算法的核心目标是通过更精细地调整置信区间上限的计算方式，来提升 UCB 算法的效率和效果。选择哪种改进版算法，需要根据具体的应用场景和性能目标来决定。简而言之，这些改进旨在通过更精准地控制探索和利用之间的平衡，来优化 UCB 算法的表现。

（3）常见常数项

在 UCB 算法的常数项中，常见的两个值是 $\sqrt{2}$ 和 $1/\sqrt{2}$ ，这两个值是根据置信区间上限的计算方法来选择。

当使用置信区间上限的标准计算公式时，常数项被设置为 $\sqrt{2}$ 。这种计算方法通常用于 MCTS 中的基本实现，这是因为根据中心极限定理，置信区间的标准差等于 $\sqrt{2}$ ， $\sqrt{2}$ 作为常数项可以确保置信区间上限正确地覆盖真实奖励的真实值。

而当使用优化的置信区间上限计算公式时，常数项被设置为 $1/\sqrt{2}$ 。这种计算方法通常用于多臂赌博机问题中，由于在这种问题中，每个动作的奖励都是独立的，因此使用 $1/\sqrt{2}$ 作为常数项可以使得算法具有更好的性能。

总的来说，常数项的选择取决于具体的应用场景和算法实现。 $\sqrt{2}$ 是 MCTS 中的主流选择，而 $1/\sqrt{2}$ 通常用于多臂赌博机问题和其他优化的 UCB 算法中。不过在实际测试中，我们发现采用 $1/\sqrt{2}$ 的结果更好。

2.4 基本算法思路

- **Node 类**：用于表示搜索树中的一个节点，它封装了节点的状态信息、被访问的次数以及累积的奖励值等关键数据。该类提供了一系列功能，例如计算节点的 UCB 值、增加子节点和判断节点是否已经完全扩展。通过这些功能，Node 类为构建和管理搜索树提供了基础架构，使得算法能够有效地进行状态空间的探索和决策。
- **选择**：从根节点开始，递归选择 UCB 值最大的一个节点（没有被扩展的节点视为 UCB 值无限大），UCB 值由 `get_uct(self, ucb_param)` 得到。

Python

```
def select(self, node):  
    """  
    :param node: 某个节点  
    :return: ucb 值最大的叶子  
    """  
  
    # print(len(node.children))  
  
    if len(node.children) == 0:    # 叶子，需要扩展  
        return node  
  
    if node.full_expanded():    # 完全扩展, 递归选择 ucb 最大的子  
节点  
        max_node = None  
        max_ucb = -sys.maxsize  
        for child in node.children:  
            child_ucb = child.get_ucb(self.ucb_param)  
            if max_ucb < child_ucb:  
                max_ucb = child_ucb  
                max_node = child    # max_node 指向 ucb 最  
大的子节点  
        return self.select(max_node)  
  
    else:    # 没有完全扩展就选访问次数为 0 的子节点  
        for kid in node.children:    # 从左开始遍历  
            if kid.visits == 0:  
                return kid
```

- 扩展：（1）我们需要确定当前节点下的所有子节点是否都被访问过，并且这些子节点都不是终止节点。如果这个条件成立，那么我们将选择具有最大 UCB 值的子节点进行扩展。这意味着我们将为这个子节点添加新的子节点，并进行初始化。然后，返

回这个被扩展的节点。（2）如果当前节点下还有未被访问的子节点，那么我们不会进行任何扩展操作。相反，我们会直接返回这个尚未被访问的子节点。

Python

```
def extend(self, node):

    if node.visits == 0:    # 自身还没有被访问过，不扩展，直接模拟
        return node

    else:    # 需要扩展,先确定颜色

        if node.color == 'X':

            new_color = 'O'

        else:

            new_color = 'X'

        for action in
list(node.now_board.get_legal_actions(node.color)):    # 把所有可行节点加入
子节点列表，并初始化

            new_board = deepcopy(node.now_board)

            new_board._move(action, node.color)

            # 新建节点

            node.add_child(new_board, action=action, color=new_color)

        if len(node.children) == 0:

            return node

        return node.children[0]    # 返回新的子节点列表的第一个，以供
下一步模拟
```

- **模拟：**从之前选定的节点出发，我们将执行一次模拟对局，这个过程将一直持续到游戏决出胜负或者达到预设的步数上限。在模拟结束后，我们将获取并返回这次模拟对局中获得的分数。
- **反向传播：**对于路径上的每个节点，我们将根据模拟对局的结果更新其值。如果模拟对局中玩家选择了特定的颜色（例如，正数代表一种颜色，负数代表另一种颜色），则节点值将相应地增加或减少新的奖励值。在传播过程中，我们还会将路径上所有节点的访问次数增加 1，以记录每个节点被访问的频率。这个过程确保了模拟对局的结果能

够反馈到搜索树中，通过调整节点值来反映新的信息，同时更新访问次数以追踪节点的探索情况。

- 最后在达到步数限制之后，我们将在搜索树的根节点的直接子节点中进行选择。具体来说，我们将评估这些第一级子节点的 UCB 值，并选择其中 UCB 值最高的节点。这个节点将被确定为下一步棋的走法。

2.5 训练过程

2.5.1 UCB 算法中的常数项的选择

UCB 算法中的常数项 c 越大搜索越深，越小重复次数越多。由于参数 $\sqrt{2}$ 经过试验效果并不佳，因此采用另一经验参数 $1/\sqrt{2}$ 。

2.5.2 最大的迭代次数

在 UCB 算法的实施过程中，除了需要调整算法中的常数项参数外，我们还需要确定模拟对局时允许的最大迭代次数。这个最大迭代次数直接关联到 AI 棋手的思考深度和计算速度。为了在确保 AI 棋手落子速度的同时，也能保持棋局得分的竞争力，我们通过一系列的实验和测试，最终决定将最大迭代次数设定为 150。这样的设置旨在平衡 AI 棋手的反应速度和决策质量。

2.5.3 最终参数

最终在 AI 玩家初始化时我们设置参数如下：

```
Python
self.max_times = 150

self.ucb_param = 1 / math.sqrt(2)
```

2.6 算法拓展

在蒙特卡洛树搜索（MCTS）算法的模拟阶段，我们最初采用了简单的随机动作选择方法。但这种方法由于其随机性，导致结果具有较大的不确定性，可能需要很长时间才能找到合适的落子点。为了提高效率，我们参考了相关资料，并决定对动作选择策略进行优化。

经过研究发现，单一的贪心策略较为短视，容易被逆风翻盘。确定子策略的算法比较复杂，程序效率较低，而机动性策略难以用程序体现，奇偶策略的效果也不佳。鉴于角点在棋局中的重要性，我们决定采用位置优先策略。这种策略为棋盘上的每一步走子都分配了一个优先级，每次走子时都从可选的走子中选择优先级最高的那一步。

常见的优先级策略主要有以下两种，左边的优先级表的首要策略是占据角点，所有优先级的设置都为此服务，右边的优先级表由 Roxanne 提出，结合了多种策略，同时也结合了 Mobility 的特性，因为中间子的优先级较高，会提高自己的 Mobility 而限制对手的可走步数。综合来看，第二个优先级表考虑更加全面，作为我们算法的首选。

	A	B	C	D	E	F	G	H
1	1	9	2	4	4	2	9	1
2	9	10	8	7	7	8	10	9
3	2	8	3	5	5	3	8	2
4	4	7	5	6	6	5	7	4
5	4	7	5	6	6	5	7	4
6	2	8	3	5	5	3	8	2
7	9	10	8	7	7	8	10	9
8	1	9	2	4	4	2	9	1

	A	B	C	D	E	F	G	H
1	1	5	3	3	3	3	5	1
2	5	5	4	4	4	4	5	5
3	3	4	2	2	2	2	4	3
4	3	4	2			2	4	3
5	3	4	2			2	4	3
6	3	4	2	2	2	2	4	3
7	5	5	4	4	4	4	5	5
8	1	5	3	3	3	3	5	1

3. 结果与讨论

3.1 胜率分析

在 MO 多智能体平台上，我们通过与各能力级别的人工智能测试对手对弈，获取了相应的测试数据。在与初级人工智能的对弈过程中，我们的算法展现了快速的决策能力，且在棋盘上占据了更多的优势棋子；而在与中级对手的较量里，决策速度有所减缓，优势棋子数量也相应缩减，胜率随之降低；在与高级人工智能对弈时，胜率更是进一步下滑。经过多次的反复测试，我们发现尽管我们的 AI 程序总体上胜率偏高，但电脑偶尔也会胜出。我们认为，这可能与棋盘的大小、对弈的先后次序以及 UCB 公式里常数 C 的选择等多种因素相关。

为了衡量胜负关系以及获胜棋子的数量，我们设定获胜积分为 10 分，每多赢得一个棋子则额外加 1 分。针对 UCB1 与 UCB2 两种算法，我们分别进行了先手和后手的十轮连续测试，并详细记录了每轮的得分数据。从下表的数据中，我们可以明显看出，UCB1 算法在测试中相较于 UCB2 算法取得了更为出色的成绩。

局数		1	2	3	4	5	6	7	8
ucb1	先手	24	44	25	27	-23	35	26	41

	后手	2	25	16	26	-10	28	19	14
ucb2	先手	26	35	40	23	27	-15	54	-20
	后手	14	17	29	28	-15	12	14	29

UCB1 算法在先手和后手的对局中均表现出 8 胜 2 负的优异成绩；而 UCB2 算法在先手时同样取得 8 胜 2 负的战绩，在后手时则实现了完美的 10 胜 0 负。从这些数据中可以观察到，UCB2 算法在后手对局中似乎具有更为显著的优势。然而，鉴于测试的次数相对有限，样本量不足，因此所得出的结论尚缺乏足够的统计学意义。尽管如此，这些初步结果激发了我们对黑白棋游戏中先手与后手优势问题的深入探讨。

3.2 相关讨论

在探讨黑白棋中先手与后手的优势问题时，网络上存在多种观点。经过资料的搜集与分析，我们得出结论：在对弈过程中，若双方均不采取跳步策略，黑棋的每次落子将导致棋盘上空位数量保持为偶数，而白棋的落子则会使空位数量变为奇数。基于此，我们可以推断，白棋在对局中将执行最终的落子，可能因此获得微弱的优势，因为其所落之子及其翻转的棋子将形成更为稳固的局势。奇偶性为白棋提供了一种固有的优势。然而，黑棋亦有策略将此转化为己方优势：通过实施奇数次的跳步，黑棋可以改变奇偶性；若再次跳步，则局势将复归原状。因此，黑棋在对弈中倾向于主动执行奇数次的跳步。黑棋控制奇偶性的有效策略是迫使白棋构建一个无法落子的奇数领域。

近年来，蒙特卡罗树搜索（MCTS）与强化学习的结合已成为研究的热点。该方法的核心在于利用神经网络来近似价值函数与策略函数，以替代传统手工设计的启发式函数。神经网络的优势在于其能够从经验中学习，从而适应各种游戏与环境，相较于传统启发式函数，此方法显著提升了搜索效率与游戏成绩。结合 MCTS 与强化学习的成功案例包括：

AlphaGo 与 AlphaGo Zero：这两个 AI 系统均采用了 MCTS 与深度强化学习的方法，在围棋等复杂游戏中展现了卓越的性能。

MuZero：MuZero 是 DeepMind 于 2020 年提出的一种基于 MCTS 与强化学习的通用游戏算法。MuZero 通过神经网络学习环境的动态模型与价值函数，并利用 MCTS 来寻找最优策略。MuZero 的优势在于能够在没有先验知识的情况下学习，并在多个游戏中取得优异成绩。

值得注意的是，将 MCTS 与强化学习融合也面临若干挑战与问题。例如，如何高效地利用神经网络来近似价值函数与策略函数，以及如何有效地将搜索与学习相结合。因此，针对这些问题的解决方案仍需进一步的研究与探索。

3.3 算法对战结果

3.3.UCB1 与 UCB2 对战

局数	1	2	3	4	5	6	7	8	9	10
ucb1 领先数	26	34	12	34	42	50	22	62	30	34

令 UCB1 算法 AI 执黑棋，UCB2 算法 AI 执白棋，对战 10 轮得到结果如上图所示，可以看出 UCB1 算法不仅略胜一筹，而且平均获胜轮得分也高于 UCB1 算法。

综合结果，我们认为 UCB1 相比于 UCB2 算法是略有优势的，这与之前让 UCB1 和 UCB2 算法 AI 分别与测试高级机器人对战的结果相同。

3.3.2 位置优先策略模拟与随机动作对战模拟

在 MTCS 的模拟环节，分别采用随机动作策略和位置优先策略模拟，让两个 AI 对战，随机动作策略执黑棋，位置优先策略执白棋，一下是几次测试结果：

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✔	108s	黑棋获胜, 领先棋子数: 34

确定

测试详情 展示棋盘

测试点	状态	时长	结果
对手对弈	✔	105s	黑棋获胜, 领先棋子数: 61

确定

测试详情

展示棋盘

×

测试点	状态	时长	结果
对手对弈	✔	112s	黑棋获胜, 领先棋子数: 22

确定

最终结果我们将其做成表格显示如下：

	A	B	C	D	E	F	G
1	X	X	X	X	X	O	X
2	O	X	X	X	O	X	X
3	X	O	X	X	X	X	X
4	O	X	O	X	X	X	O
5	X	X	X	X	O	X	X
6	X	X	X	X	X	X	X
7	X	X	O	X	X	X	X

从对战结果可以看出，随机动作策略比位置优先策略，获胜概率更大。

4. 总结

通过实验实现了基于蒙特卡洛树搜索算法（MCTS）的 miniAlphaGo 用于黑白棋对弈。实验结果显示，该算法能够有效与不同等级的 AI 进行对抗，并在某些条件下获得较高的胜率，尤其是后手的情况下。因此，可以说达到了实验的基本目标。

在实现过程中遇到了一些挑战：初始阶段使用了完全随机的模拟策略，但这种策略导致结果的不确定性较高，难以稳定提高胜率。在尝试了位置优先策略后，虽然在某些情形下改进效果明显，但仍存在因超时而无法得到有效结果的情况。

目前使用的位置优先策略在部分情况下超时未能完成模拟。可以进一步结合启发式策略与强化学习技术来改进模拟的效率。例如，采用深度神经网络来近似策略函数和价值函数，以替代简单的启发式函数。此外，在 UCB1 和 UCB2 的算法测试中，发现不同的平衡策略带来了不同的效果。在未来，可以尝试更多 UCB 改进算法，如 UCB-Tuned，以提高搜索效率。

附录 1：完整实验代码-随机动作策略模拟

```
Python
import math
import random
import sys
from game import Game # 导入黑白棋文件
from copy import deepcopy

class Node:
    def __init__(self, now_board, parent=None, action=None,
color=""):
        self.visits = 0 # 访问次数
        self.reward = 0.0 # 期望值
        self.now_board = now_board # 棋盘状态
        self.children = [] # 子节点
        self.parent = parent # 父节点
        self.action = action # 对应动作
        self.color = color # 该节点玩家颜色
    def get_ucb(self, ucb_param):
        if self.visits == 0:
            return sys.maxsize # 未访问的节点 ucb 为无穷大
        # UCB 公式
        explore = math.sqrt(2.0 * math.log(1+self.parent.visits) /
float(1+self.visits))
        now_ucb = self.reward/self.visits + ucb_param * explore
        return now_ucb
    def add_child(self, child_now_board, action, color):
        child_node = Node(child_now_board, parent=self,
action=action, color=color)
        self.children.append(child_node)
    # 判断是否完全扩展
    def full_expanded(self):
        # 有子节点并且所有子节点都访问过了就是完全扩展
        if len(self.children) == 0:
            return False
```

```

        for kid in self.children:
            if kid.visits == 0:
                return False

        return True

class AIPlayer:
    """
    AI 玩家
    """
    def __init__(self, color):
        """
        玩家初始化
        :param color: 下棋方, 'X' - 黑棋, 'O' - 白棋
        """

        self.max_times = 150 # 最大迭代次数
        self.ucb_param = 0.707 # ucb 的参数 C
        self.color = color

    def uct(self, max_times, root):
        """
        根据当前棋盘状态获取最佳落子位置
        :param max_times: 最大搜索次数
        :param root: 根节点
        :return: action 最佳落子位置
        """

        for i in range(max_times): # 最多模拟 max 次
            selected_node = self.select(root)
            leaf_node = self.extend(selected_node)
            reward = self.stimulate(leaf_node)
            self.backup(leaf_node, reward)

        max_node = None # 搜索完成, 然后找出最适合的下一步
        max_ucb = -sys.maxsize
        for child in root.children:
            child_ucb = child.get_ucb(self.ucb_param)
            if max_ucb < child_ucb:
                max_ucb = child_ucb

```



```

        max_node = child # max_node 指向 ucb 最大的子节点

    return max_node.action

def select(self, node):
    """
    :param node: 某个节点
    :return: ucb 值最大的叶子
    """
    # print(len(node.children))
    if len(node.children) == 0: # 叶子，需要扩展
        return node
    if node.full_expanded(): # 完全扩展，递归选择 ucb 最大的子节点
        max_node = None
        max_ucb = -sys.maxsize
        for child in node.children:
            child_ucb = child.get_ucb(self.ucb_param)
            if max_ucb < child_ucb:
                max_ucb = child_ucb
                max_node = child # max_node 指向 ucb 最大的子节点
        return self.select(max_node)

    else: # 没有完全扩展就选访问次数为 0 的子节点
        for kid in node.children: # 从左开始遍历
            if kid.visits == 0:
                return kid

def extend(self, node):
    if node.visits == 0: # 自身还没有被访问过，不扩展，直接模拟
        return node
    else: # 需要扩展，先确定颜色
        if node.color == 'X':
            new_color = 'O'
        else:
            new_color = 'X'
        for action in
list(node.now_board.get_legal_actions(node.color)):

```

```

        new_board = deepcopy(node.now_board)
        new_board._move(action, node.color)
        # 新建节点
        node.add_child(new_board, action=action,
color=new_color)
    if len(node.children) == 0:
        return node
    return node.children[0]    # 返回新的子节点列表的第一个，
以供下一步模拟

def stimulate(self, node):
    """
    :param node:模拟起始点
    :return: 模拟结果 reward
    board.get_winner()会返回胜负关系和获胜子数
    考虑胜负关系和获胜的子数，定义获胜积 10 分，每多赢一个棋子多 1 分
    """
    board = deepcopy(node.now_board)
    color = node.color
    count = 0
    while (not self.game_over(board)) and count < 50:    # 游戏没
有结束，就模拟下棋
        action_list =
list(node.now_board.get_legal_actions(color))
        if not len(action_list) == 0:    # 可以下，就随机下棋
            action = random.choice(action_list)
            board._move(action, color)
            if color == 'X':
                color = 'O'
            else:
                color = 'X'
        else:    # 不能下，就交换选手
            if color == 'X':
                color = 'O'
            else:
                color = 'X'
        action_list =

```

```

list(node.now_board.get_legal_actions(color))
        action = random.choice(action_list)
        board._move(action, color)
        if color == 'X':
            color = 'O'
        else:
            color = 'X'
        count = count + 1

# winner:0-黑棋赢, 1-白棋赢, 2-表示平局
# diff:赢家领先棋子数
winner, diff = board.get_winner()
if winner == 2:
    reward = 0
elif winner == 0:
    # 这逻辑
    reward = 10 + diff
else:
    reward = -(10 + diff)

if self.color == 'X':
    reward = - reward

return reward

def backup(self, node, reward):
    """
    反向传播函数
    """
    while node is not None:
        node.visits += 1
        if node.color == self.color:
            node.reward += reward
        else:
            node.reward -= reward
        node = node.parent
    return 0

```

```

def game_over(self, board):
    """
    判断游戏是否结束
    :return: True/False 游戏结束/游戏没有结束
    """
    # 根据当前棋盘，双方都无处可落子，则终止
    b_list = list(board.get_legal_actions('X'))
    w_list = list(board.get_legal_actions('O'))
    is_over = (len(b_list) == 0 and len(w_list) == 0) # 返回值
    True/False

    return is_over

def get_move(self, board):
    """
    根据当前棋盘状态获取最佳落子位置
    :param board: 棋盘
    :return: action 最佳落子位置, e.g. 'A1'
    """
    if self.color == 'X':
        player_name = '黑棋'
    else:
        player_name = '白棋'
    print("请等一会, 对方 {}-{} 正在思考中...".format(player_name,
self.color))
    root = Node(now_board=deepcopy(board), color=self.color)
    action = self.uct(self.max_times, root)

    return action

```

附录 2：完整实验代码-位置优先策略模拟

```

import math
import random
import sys
from game import Game
from copy import deepcopy
import datetime

class Node:
    def __init__(self, now_board, parent=None, action=None,
color=""):
        self.visits = 0 # 访问次数
        self.reward = 0.0 # 期望值
        self.now_board = now_board # 棋盘状态
        self.children = [] # 孩子节点
        self.parent = parent # 父节点
        self.action = action # 对应动作
        self.color = color # 该节点玩家颜色

    def get_ucb(self, ucb_param):
        if self.visits == 0:
            return sys.maxsize # 未访问的节点 ucb 为无穷大

        # UCB2 公式
        explore = math.sqrt(2.0 * math.log(1+self.parent.visits) /
float(1+self.visits))
        now_ucb = self.reward/self.visits + ucb_param * explore
        return now_ucb

    # 增加子节点
    def add_child(self, child_now_board, action, color):
        child_node = Node(child_now_board, parent=self,
action=action, color=color)
        self.children.append(child_node)

    # 判断是否完全扩展
    def full_expanded(self):
        # 有子节点并且所有子节点都访问过了就是完全扩展
        if len(self.children) == 0:
            return False
        for kid in self.children:
            if kid.visits == 0:
                return False

        return True

```

```

class SilentGame(object):
    ''' 重构游戏类，模拟下棋过程中，不实时打印棋盘 '''

    def __init__(self, black_player, white_player, board,
current_player=None):
        self.board = deepcopy(board) # 棋盘
        # 定义棋盘上当前下棋棋手，先默认是 None
        self.current_player = current_player
        self.black_player = black_player # 黑棋一方
        self.white_player = white_player # 白棋一方
        self.black_player.color = "X"
        self.white_player.color = "O"

    def switch_player(self, black_player, white_player):
        """
        游戏过程中切换玩家
        :param black_player: 黑棋
        :param white_player: 白棋
        :return: 当前玩家
        """

        # 如果当前玩家是 None 或者 白棋一方 white_player，则返回 黑
        棋一方 black_player;
        if self.current_player is None:
            return black_player
        else:
            # 如果当前玩家是黑棋一方 black_player 则返回 白棋一方
            white_player
            if self.current_player == self.black_player:
                return white_player
            else:
                return black_player

    def print_winner(self, winner):
        """
        打印赢家
        :param winner: [0,1,2] 分别代表黑棋获胜、白棋获胜、平局 3 种
        可能。
        :return:
        """

        print(['黑棋获胜!', '白棋获胜!', '平局'][winner])

    def force_loss(self, is_timeout=False, is_board=False,
is_legal=False):

```

```

"""
    落子 3 个不符合规则和超时则结束游戏, 修改棋盘也是输
:param is_timeout: 时间是否超时, 默认不超时
:param is_board: 是否修改棋盘
:param is_legal: 落子是否合法
:return: 赢家 (0,1), 棋子差 0
"""

if self.current_player == self.black_player:
    win_color = '白棋 - 0'
    loss_color = '黑棋 - X'
    winner = 1
else:
    win_color = '黑棋 - X'
    loss_color = '白棋 - 0'
    winner = 0

if is_timeout:
    print('\n{} 思考超过 60s, {} 胜'.format(loss_color,
win_color))
    if is_legal:
        print('\n{} 落子 3 次不符合规则, 故 {} 胜
'.format(loss_color, win_color))
    if is_board:
        print('\n{} 擅自改动棋盘判输, 故 {} 胜
'.format(loss_color, win_color))

    diff = 0

    return winner, diff

def run(self):
    """
    运行游戏
    :return:
    """
    # 定义统计双方下棋时间
    total_time = {"X": 0, "O": 0}
    # 定义双方每一步下棋时间
    step_time = {"X": 0, "O": 0}
    # 初始化胜负结果和棋子差
    winner = None
    diff = -1

```

```

        # 游戏开始
        while True:
            # 切换当前玩家, 如果当前玩家是 None 或者白棋
            white_player, 则返回黑棋 black_player;
            # 否则返回 white_player。
            self.current_player =
            self.switch_player(self.black_player, self.white_player)
            start_time = datetime.datetime.now()
            # 当前玩家对棋盘进行思考后, 得到落子位置
            # 判断当前下棋方
            color = "X" if self.current_player == self.black_player
            else "O"
            # 获取当前下棋方合法落子位置
            legal_actions =
            list(self.board.get_legal_actions(color))
            # print("%s 合法落子坐标列表: "%color, legal_actions)
            if len(legal_actions) == 0:
                # 判断游戏是否结束
                if self.game_over():
                    # 游戏结束, 双方都没有合法位置
                    winner, diff = self.board.get_winner() # 得到赢
                    家 0,1,2
                    break
                else:
                    # 另一方有合法位置, 切换下棋方
                    continue

            action = self.current_player.get_move(self.board)

            if action is None:
                continue
            else:
                self.board._move(action, color)
                if self.game_over():
                    winner, diff = self.board.get_winner() # 得到赢
                    家 0,1,2
                    break

            return winner, diff

    def game_over(self):
        """
        判断游戏是否结束
        :return: True/False 游戏结束/游戏没有结束

```



```

"""

# 根据当前棋盘，判断棋局是否终止
# 如果当前选手没有合法下棋的位子，则切换选手；如果另外一个选手也没有合法的下棋位置，则比赛停止。
b_list = list(self.board.get_legal_actions('X'))
w_list = list(self.board.get_legal_actions('O'))

is_over = len(b_list) == 0 and len(w_list) == 0 # 返回值
True/False

return is_over

class AIPlayer:

    def __init__(self, color):
        """
        玩家初始化
        :param color: 下棋方，'X' - 黑棋，'O' - 白棋
        """

        self.max_times = 5 # 最大迭代次数
        self.uch_param = 1/math.sqrt(2) # ucb 的参数 C

        self.color = color
        self.roxanne_table = [
            ['A1', 'H1', 'A8', 'H8'],
            ['C3', 'F3', 'C6', 'F6'],
            ['C4', 'F4', 'C5', 'F5', 'D3', 'E3', 'D6', 'E6'],
            ['A3', 'H3', 'A6', 'H6', 'C1', 'F1', 'C8', 'F8'],
            ['A4', 'H4', 'A5', 'H5', 'D1', 'E1', 'D8', 'E8'],
            ['B3', 'G3', 'B6', 'G6', 'C2', 'F2', 'C7', 'F7'],
            ['B4', 'G4', 'B5', 'G5', 'D2', 'E2', 'D7', 'E7'],
            ['B2', 'G2', 'B7', 'G7'],
            ['A2', 'H2', 'A7', 'H7', 'B1', 'G1', 'B8', 'G8']
        ]
        """

        self.roxanne_table = [
            ['A1', 'H1', 'A8', 'H8'],
            ['C3', 'D3', 'E3', 'F3', 'C4', 'F4'],
            ['C5', 'F5', 'C6', 'D6', 'E6', 'F6'],
            ['C1', 'D1', 'E1', 'F1', 'C8', 'D8', 'E8', 'F8'],
            ['A3', 'A4', 'A5', 'A6', 'H3', 'H4', 'H5', 'H6'],
            ['C4', 'D4', 'E4', 'F4', 'C7', 'D7', 'E7', 'F7'],
            ['B3', 'B4', 'B5', 'B6', 'G3', 'G4', 'G5', 'G6'],

```

```

        ['A2', 'A7', 'B1', 'B2', 'B7', 'B8'],
        ['G1', 'G2', 'G7', 'G8', 'H2', 'H7']
    ]

"""

def uct(self, max_times, root):
    """
    根据当前棋盘状态获取最佳落子位置
    :param max_times: 最大搜索次数
    :param root: 根节点
    :return: action 最佳落子位置
    """

    for i in range(max_times): # 最多模拟 max 次
        selected_node = self.select(root)
        leaf_node = self.extend(selected_node)
        reward = self.stimulate(leaf_node)
        self.backup(leaf_node, reward)

    max_node = None # 搜索完成, 然后找出最适合的下一步
    max_ucb = -sys.maxsize
    for child in root.children:
        child_ucb = child.get_ucb(self.ucb_param)
        if max_ucb < child_ucb:
            max_ucb = child_ucb
            max_node = child # max_node 指向 ucb 最大的子节点

    return max_node.action

def select(self, node):
    """
    :param node: 某个节点
    :return: ucb 值最大的叶子
    """

    # print(len(node.children))
    if len(node.children) == 0: # 叶子, 需要扩展
        return node
    if node.full_expanded(): # 完全扩展, 递归选择 ucb 最大的子
节点
        max_node = None
        max_ucb = -sys.maxsize
        for child in node.children:
            child_ucb = child.get_ucb(self.ucb_param)
            if max_ucb < child_ucb:

```

节点

```
        max_uct = child_uct
        max_node = child    # max_node 指向 uct 最大的子

    return self.select(max_node)

else:    # 没有完全扩展就选访问次数为 0 的子节点
    for kid in node.children:    # 从左开始遍历
        if kid.visits == 0:
            return kid

def extend(self, node):
    if node.visits == 0:    # 自身还没有被访问过，不扩展，直接模拟
        return node
    else:    # 需要扩展, 先确定颜色
        if node.color == 'X':
            new_color = 'O'
        else:
            new_color = 'X'
        for action in
list(node.now_board.get_legal_actions(node.color)):    # 把所有可行节点加入子节点列表，并初始化
            new_board = deepcopy(node.now_board)
            new_board._move(action, node.color)
            # 新建节点
            node.add_child(new_board, action=action,
color=new_color)
            if len(node.children) == 0:
                return node
            return node.children[0]    # 返回新的子节点列表的第一个，以供下一步模拟

def stimulate(self, node):
    """
    :param node: 模拟起始点
    :return: 模拟结果 reward
    board.get_winner() 会返回胜负关系和获胜子数
    考虑胜负关系和获胜的子数，定义获胜积 10 分，每多赢一个棋子多
    1 分
    """

    """
    蒙特卡洛树搜索，采用 Roxanne 策略代替随机策略搜索，模拟扩展搜索树
```

索树

```

"""
board = deepcopy(node.now_board)
color = node.color
count = 0
while (not self.game_over(board)) and count < 50:    # 游戏没
有结束，就模拟下棋
    action_list =
list(node.now_board.get_legal_actions(color))
    if not len(action_list) == 0:    # 可以下，就随机下棋
        for move_list in self.roxanne_table:
            random.shuffle(move_list)
            for move in move_list:
                if move in action_list:
                    action = move
                    break
            break
        board._move(action, color)
        if color == 'X':
            color = 'O'
        else:
            color = 'X'
    else:    # 不能下，就交换选手
        if color == 'X':
            color = 'O'
        else:
            color = 'X'
    action_list =
list(node.now_board.get_legal_actions(color))
    for move_list in self.roxanne_table:
        random.shuffle(move_list)
        for move in move_list:
            if move in action_list:
                action = move
                break
        break
    board._move(action, color)
    if color == 'X':
        color = 'O'
    else:
        color = 'X'
    count = count + 1

# winner:0-黑棋赢，1-白旗赢，2-表示平局
# diff:赢家领先棋子数

```

```

        if winner == 2:
            reward = 0
        elif winner == 0:
            reward = 10 + diff
        else:
            reward = -(10 + diff)

        if self.color == 'X':
            reward = - reward

    return reward

def backup(self, node, reward):
    """
    反向传播函数
    """
    while node is not None:
        node.visits += 1
        if node.color == self.color:
            node.reward += reward
        else:
            node.reward -= reward
        node = node.parent
    return 0

def game_over(self, board):

    # 根据当前棋盘，双方都无处可落子，则终止
    b_list = list(board.get_legal_actions('X'))
    w_list = list(board.get_legal_actions('O'))
    is_over = (len(b_list) == 0 and len(w_list) == 0) # 返回值
    True/False

    return is_over

def get_move(self, board):
    if self.color == 'X':
        player_name = '黑棋'
    else:
        player_name = '白棋'
    print("请等一会，对方 {}-{} 正在思考
    中...".format(player_name, self.color))

    root = Node(now_board=deepcopy(board), color=self.color)

```

```
    action = self.uct(self.max_times, root)

    return action
```